



廣州軟件學院
GUANGZHOU UNIVERSITY OF SOFTWARE

实验报告封面

课程名称: Web应用项目实训 课程代码: CW3104

任课老师: 包玉琳 实验指导老师: 包玉琳

实验报告名称: 实验4 安全管理

本实验报告包括以下几个内容:

- 一、实验目的 ☐
- 二、实验环境 ☐
- 三、实验准备 ☐
- 四、实验过程 ☐

学生姓名:

学号:

教学班:

递交日期:

签收人:

我申明,本报告内的实验已按要求完成,报告完全是由我个人完成,并没有抄袭行为。我已经保留了这份实验报告的副本。

申明人(签名):

实验报告评语与评分:

评阅老师签名:

一、实验目的

能够说明认证与授权的区别,能区分 401 与 403 的语义、触发场景与对应的处理组件。

能够说明 JWT 的三段结构,理解「令牌载荷只放用户 ID、完整用户信息存入 Redis」这一设计的理由。

能够按步骤完成 Spring Security 认证链的搭建:引入依赖、配置 Redis 序列化、改造 JWT 工具类、编写用户详情服务与用户详情模型、编写安全配置类、实现登录接口、实现 JWT 认证过滤器、实现退出登录。

能够设计菜单表与角色菜单关联表,用一张菜单表承载目录、菜单、按钮三类数据,并遵循「模块:操作」的权限标识命名规范。

能够使用两次遍历加映射表的方式构建菜单树,并说明为什么不用递归实现。

能够实现角色列表携带菜单信息，以及角色菜单授权（先删后插、事务保护、父级目录补全）。

能够实现认证异常处理器与无权限异常处理器，完成 401 与 403 的分流返回。

能够实现权限判断服务与自定义权限管理器，完成路径级与方法级两层授权。

能够按验证清单独立完成认证与授权功能的自测。

二、实验环境

操作系统：Windows 11

开发工具：IntelliJ IDEA 2025.1.2

JDK 版本：JDK 17（本机实际以 JDK 21 运行，向下兼容）

构建工具：Maven 3.9.12

版本管理：Git + Gitee 远程仓库

数据库：MySQL 8.0

缓存数据库：Redis 5.0.10（Windows 版，本次实验新增）

数据库客户端：Navicat Premium 17；缓存客户端：redis-cli

接口测试工具：Apifox（本机未安装，使用系统自带 curl 完成等效验证）

核心依赖：Spring Boot 2.7.14、Spring Security 5.7.10、Spring Data Redis、MyBatis-Plus 3.5.3.1、Druid、MySQL Driver、Hutool 5.8.25、fastjson 1.2.83、Lombok

三、实验准备

课设项目「广软广播站点歌与投稿系统」为单模块 Spring Boot 工程，能够正常编译、启动（端口 8089，context-path 为 /api）。

已创建数据库 broadcast_db，并完成业务表与初始数据的导入，学生、广播站成员等账号均已存在。

已安装 Redis 5.0.10 并启动服务（默认端口 6379），能够使用 redis-cli 查看键与值。

已在工程 pom.xml 中引入 Spring Security、Spring Data Redis、fastjson、Hutool 等依赖。

已配置 Git 全局用户名与邮箱，能够在项目根目录提交代码（提交操作自行完成）。

已能使用 curl 发送带请求头的 HTTP 请求，并能查看响应状态码与响应体。

四、实验过程

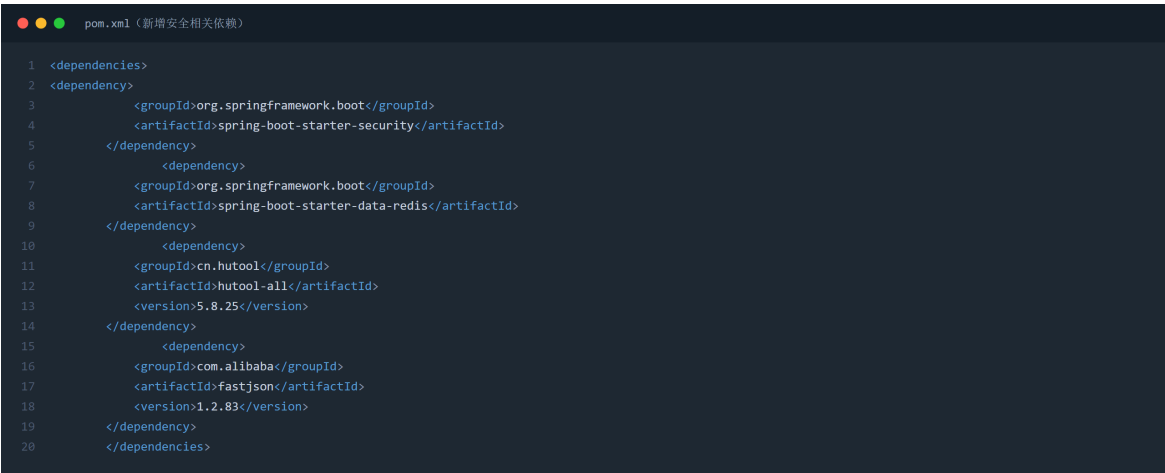
任务一：搭建认证链（Spring Security + JWT）

步骤1：引入依赖

按下表把依赖加入本工程 pom.xml；Spring Boot 启动器与 Redis 启动器版本由 Spring Boot 父工程统一管理，Hutool、fastjson 写明版本号。

组件	用途	放置模块
Spring Security 启动器	提供过滤器链、认证与授权能力	本工程
Redis 启动器	存储登录用户的完整信息	本工程
fastjson	把缓存对象序列化为可读的 JSON	本工程
Hutool（JWT）	签发与校验 JWT 令牌	本工程

截图：



```
1 <dependencies>
2 <dependency>
3     <groupId>org.springframework.boot</groupId>
4     <artifactId>spring-boot-starter-security</artifactId>
5 </dependency>
6 <dependency>
7     <groupId>org.springframework.boot</groupId>
8     <artifactId>spring-boot-starter-data-redis</artifactId>
9 </dependency>
10 <dependency>
11     <groupId>cn.hutool</groupId>
12     <artifactId>hutool-all</artifactId>
13     <version>5.8.25</version>
14 </dependency>
15 <dependency>
16     <groupId>com.alibaba</groupId>
17     <artifactId>fastjson</artifactId>
18     <version>1.2.83</version>
19 </dependency>
20 </dependencies>
```

图1 pom.xml 中新增的安全相关依赖

步骤2：配置Redis序列化

新建Redis配置类，注册一个名为redisTemplate的 Bean，按四项要求设置：键用字符串序列化器、值用 JSON 序列化器、序列化时写入类型信息、开启可信包白名单。

再新建一个缓存操作工具类，对外提供四类方法：按键写入并设置过期时间、按键读取、按键删除、为已存在的键续期。

易错点：自定义的模板必须注册成名为 redisTemplate 的 Bean，否则注入点拿到的仍是框架默认模板，序列化配置形同虚设。键名建议统一使用固定前缀加用户 ID 的形式，例如 login:user:1。

截图：（Redis客户端中查看到的缓存键与JSON 值）

```
config/RedisConfig.java

1 package com.grbroadcast.config;
2
3 import com.alibaba.fastjson.parser.ParserConfig;
4 import com.alibaba.fastjson.support.spring.GenericFastJsonRedisSerializer;
5 import org.springframework.context.annotation.Bean;
6 import org.springframework.context.annotation.Configuration;
7 import org.springframework.data.redis.connection.RedisConnectionFactory;
8 import org.springframework.data.redis.core.RedisTemplate;
9 import org.springframework.data.redis.serializer.StringRedisSerializer;
10
11 /**
12  * Redis 配置：键使用字符串序列化器，值使用 fastjson 序列化器
13  * （序列化时写入类型信息，并配置可信包白名单）
14  */
15 @Configuration
16 public class RedisConfig {
17
18     static {
19         // 可信包白名单：仅允许反序列化本项目包中的类
20         ParserConfig.getGlobalInstance().addAccept("com.grbroadcast.");
21     }
22
23     @Bean
24     public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory connectionFactory) {
25         RedisTemplate<String, Object> template = new RedisTemplate<>();
26         template.setConnectionFactory(connectionFactory);
27
28         StringRedisSerializer stringSerializer = new StringRedisSerializer();
29         // 值序列化器：写入 @type 类型信息（白名单在静态块中配置）
30         GenericFastJsonRedisSerializer jsonSerializer = new GenericFastJsonRedisSerializer();
31
32         template.setKeySerializer(stringSerializer);
33         template.setHashKeySerializer(stringSerializer);
34         template.setValueSerializer(jsonSerializer);
35         template.setHashValueSerializer(jsonSerializer);
36         template.afterPropertiesSet();
37         return template;
38     }
39 }
40
```

图2 RedisConfig 序列化配置

```
Windows PowerShell
==== 场景6: Redis 中查看登录缓存 ====
PS> redis-cli keys "login:user:*"
1) "login:user:5"
2) "login:user:4"

PS> redis-cli get login:user:5
"{\"@type\":\"com.grbroadcast.security.LoginUser\",\"accountNonExpired\":true,\"accountNonLocked\":true,\"credentialsNonExpired\":true,\"enabled\":true,\"password\":\"$2a$10$Tudjcy11b.FMhjn1b09mTe13uyCL0MfYsULzB0j1GtZ0sIF3i4Xe\",\"permissions\":Set[\"article:edit\",\"audio:upload\",\"program:list\",\"article:audit\",\"article:add\",\"song:audit\",\"audio:delete\",\"message:add\",\"menu:list\",\"song:add\",\"audio:list\",\"program:publish\",\"article:list\",\"message:reply\",\"program:edit\",\"song:delete\",\"article:delete\",\"song:list\",\"menu:grant\",\"message:list\",\"song:edit\",\"program:delete\"],\"roles\":Set[\"staff\"],\"status\":1,\"userId\":5,\"username\":\"staff1\"}"
PS C:\WINDOWS\System32>
```

图3 redis-cli 中查看到的缓存键与 JSON 值

步骤3：改造JWT工具类

工具类对外明确四项能力：签发令牌、校验令牌、从令牌中解析用户 ID、读取令牌有效期秒数。

按下表调整令牌载荷内容，改造后载荷只保留三项：用户ID、签发时间、过期时间。

注意：令牌载荷只做 Base64 编码，任何人都能解开查看，因此绝对不能放密码、手机号等隐私数据。

截图：（解开一枚令牌，展示载荷内容确实只有用户 ID 与时间信息）

```
utils/JwtUtil.java

1 package com.grbroadcast.utils;
2
3 import cn.hutool.jwt.JWT;
4 import cn.hutool.jwt.JWTUtil;
5 import org.springframework.beans.factory.annotation.Value;
6 import org.springframework.stereotype.Component;
7
8 import java.nio.charset.StandardCharsets;
9 import java.util.HashMap;
10 import java.util.Map;
11
12 /**
13  * JWT 工具类：签发令牌、校验令牌、解析用户 ID、读取有效期
14  * 载荷只保留三项：userId、签发时间、过期时间
15  */
16 @Component
17 public class JwtUtil {
18
19     @Value("${jwt.secret}")
20     private String secret;
21
22     /** 令牌有效期（秒） */
23     @Value("${jwt.expire}")
24     private long expire;
25
26     /** 签发令牌，载荷中只放用户 ID（Hutool 自动写入 iat，过期时间通过有效期校验） */
27     public String create(Long userId) {
28         Map<String, Object> payload = new HashMap<>();
29         payload.put("userId", userId);
30         long now = System.currentTimeMillis();
31         payload.put("iat", now / 1000);
32         payload.put("exp", (now + expire * 1000) / 1000);
33         return JWTUtil.createToken(payload, secret.getBytes(StandardCharsets.UTF_8));
34     }
35
36     /** 校验令牌签名与有效期 */
37     public boolean verify(String token) {
38         try {
39             JWT jwt = JWT.of(token).setKey(secret.getBytes(StandardCharsets.UTF_8));
40             if (!jwt.verify()) {
41                 return false;
42             }
43             Long exp = Long.valueOf(jwt.getPayload("exp").toString());
44             return exp >= System.currentTimeMillis() / 1000;
45         } catch (Exception e) {
46             return false;
47         }
48     }
49
50     /** 从令牌中解析用户 ID */
51     public Long getUserId(String token) {
52         Object userId = JWT.of(token).getPayload("userId");
53         return Long.valueOf(userId.toString());
54     }
55
56     /** 读取令牌有效期秒数 */
57     public long getExpire() {
58         return expire;
59     }
60 }
61
```

图4 JwtUtil 工具类

```
Windows PowerShell
==== 场景7: 解开令牌, 载荷仅含 userId/iat/exp ====
PS> JWT =
eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJleHAiOjE3OTZlODM5InVzZXJzCI6NSWiaWF0IjoxNzgzMzg1OTgzfQ._5J-IimDSQgEwYvuQ6YyYIJu6drkKOCYjdszewc0g00
PS> 解码后的载荷:
{
  "exp": 1791389583,
  "userId": 5,
  "iat": 1791385983
}
PS C:\WINDOWS\System32>
```

图5 解开令牌后载荷仅含 **userId**、**iat**、**exp**

步骤4: 编写用户详情服务与用户详情模型

用户详情服务实现 `UserDetailsService` 接口, 重写 `loadUserByUsername` 方法, 按六步完成: 按登录账号查询用户(过滤逻辑删除的数据)、判断账号是否存在、判断账号是否被禁用、查询该用户的角色编码集合、判断是否为超级管理员(是则权限集合直接放入通配符, 否则按用户 ID 查询权限标识集合)、装配成用户详情对象返回。

用户详情模型实现 `UserDetails` 接口, 至少包含六个部分: 用户 ID、登录账号、密码、账号状态、角色编码集合、权限标识集合。

两个必须注意的点: 第一, 权限为空时要返回空集合而不是空对象, 避免后续比对时层层判空; 第二, 这个类作为缓存对象必须提供无参构造方法, 反序列化时会先创建空对象再逐字段赋值。

说明: 本系统角色固定为学生 `student`、广播站成员 `staff`、指导老师 `teacher` 三类, 不设管理员账号, 代码中保留超级管理员通配符分支; 测试账号 `staff1`、`student1`, 密码均为 123456, 密码以 BCrypt 密文存储。

截图: (用户详情服务与用户详情模型的核心代码)

```

security/LoginUser.java

1 package com.grbroadcast.security;
2
3 import com.fasterxml.jackson.annotation.JsonIgnore;
4 import com.alibaba.fastjson.annotation.JSONField;
5 import lombok.Data;
6 import org.springframework.security.core.GrantedAuthority;
7 import org.springframework.security.core.authority.SimpleGrantedAuthority;
8 import org.springframework.security.core.userdetails.UserDetails;
9
10 import java.util.HashSet;
11 import java.util.Set;
12
13 /**
14  * 登录用户详情模型（同时作为 Redis 缓存对象，必须提供无参构造方法）
15  */
16 @Data
17 public class LoginUser implements UserDetails {
18
19     private static final long serialVersionUID = 1L;
20
21     /** 用户 ID */
22     private Long userId;
23
24     /** 登录账号 */
25     private String username;
26
27     /** 密码 (BCrypt 密文) */
28     private String password;
29
30     /** 账号状态: 0 禁用 1 启用 */
31     private Integer status;
32
33     /** 角色编码集合 */
34     private Set<String> roles = new HashSet<>();
35
36     /** 权限标识集合 */
37     private Set<String> permissions = new HashSet<>();
38
39     public LoginUser() {
40     }
41
42     @JsonIgnore
43     @JSONField(serialize = false)
44     @Override
45     public Set<GrantedAuthority> getAuthorities() {
46         Set<GrantedAuthority> authorities = new HashSet<>();
47         for (String role : roles) {
48             authorities.add(new SimpleGrantedAuthority("ROLE_" + role));
49         }
50         for (String permission : permissions) {
51             authorities.add(new SimpleGrantedAuthority(permission));
52         }
53         return authorities;
54     }
55
56     @JsonIgnore
57     @Override
58     public boolean isAccountNonExpired() {
59         return true;
60     }
61
62     @JsonIgnore
63     @Override
64     public boolean isAccountNonLocked() {
65         return true;
66     }
67
68     @JsonIgnore
69     @Override
70     public boolean isCredentialsNonExpired() {
71         return true;
72     }
73
74     @JsonIgnore
75     @Override
76     public boolean isEnabled() {
77         return status != null && status == 1;
78     }
79 }
80

```

图6 LoginUser 用户详情模型

```

security/UserDetailsServiceImpl.java

1 package com.grbroadcast.security;
2
3 import com.grbroadcast.dao.MenuMapper;
4 import com.grbroadcast.dao.UserMapper;
5 import com.grbroadcast.entity.User;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.security.authentication.DisabledException;
8 import org.springframework.security.core.userdetails.UsernameNotFoundException;
9 import org.springframework.stereotype.Service;
10
11 import java.util.Collections;
12 import java.util.HashSet;
13 import java.util.Set;
14
15 /**
16  * 用户详情服务：按账号查询用户、角色与权限，装配为 LoginUser
17  */
18 @Service
19 public class UserDetailsServiceImpl implements org.springframework.security.core.userdetails.UserDetailsService {
20
21     @Autowired
22     private UserMapper userMapper;
23
24     @Autowired
25     private MenuMapper menuMapper;
26
27     @Override
28     public LoginUser loadUserByUsername(String username) {
29         // 1. 按登录账号查询用户
30         User user = userMapper.findByUsername(username);
31         // 2. 判断账号是否存在
32         if (user == null) {
33             throw new UsernameNotFoundException("用户不存在");
34         }
35         // 3. 判断账号是否被禁用
36         if (user.getStatus() == null || user.getStatus() != 1) {
37             throw new DisabledException("账号已被禁用");
38         }
39         // 4. 查询角色编码集合
40         Set<String> roles = new HashSet<>(Collections.singletonList(user.getRole()));
41         // 5. 超级管理员放入通配符，否则按角色查询权限标识集合
42         Set<String> permissions;
43         if (roles.contains("admin")) {
44             permissions = new HashSet<>(Collections.singletonList("*:~:*"));
45         } else {
46             permissions = menuMapper.selectPermsByRole(user.getRole());
47             if (permissions == null) {
48                 permissions = new HashSet<>();
49             }
50         }
51         // 6. 装配用户详情对象
52         LoginUser loginUser = new LoginUser();
53         loginUser.setUserId(user.getId());
54         loginUser.setUsername(user.getUsername());
55         loginUser.setPassword(user.getPassword());
56         loginUser.setStatus(user.getStatus());
57         loginUser.setRoles(roles);
58         loginUser.setPermissions(permissions);
59         return loginUser;
60     }
61 }
62

```

图7 UserDetailsServiceImpl 用户详情服务

步骤5：编写安全配置类

新建安全配置类，注册 SecurityFilterChain 类型的 Bean（Spring Security 5.7 起 WebSecurityConfigurerAdapter 已被弃用），并按表中逐项完成设置。

配置项	设置要求	说明
跨站请求伪造防护	关闭	前后端分离项目用令牌认证，不依赖会话
表单登录	关闭	不关闭时未认证请求会被重定向到登录页
HTTP 基础认证	关闭	同上，避免弹出浏览器原生登录框
方法级安全注解	开启	不开启时权限注解会静默失效，不报错也不警告
框架自带退出接口	关闭	自带接口行为固定，需自行实现退出逻辑
跨域配置	迁移到此处	由安全框架统一接管跨域
预检请求	放行 OPTIONS 方法	预检请求不携带令牌，拦截会导致跨域失败
白名单	放行登录、注册、健康检查、错误页转发路径	否则会出现「想登录但登录接口也被拦」
其余请求	要求已认证	白名单放行、其余全拦
密码编码器	注册 BCrypt	认证过程中框架会自动调用
JWT 过滤器	注册并置于用户名密码认证过滤器之前	顺序错误会导致后续授权环节读不到身份

同时注册一个 `AuthenticationManager` 的 Bean 供登录接口调用。

版本适配说明：本工程实际使用 `Spring Boot 2.7.14`、`Spring Security 5.7.10`。与模板基于的 `Spring Boot 3.2 / Security 6` 相比，URL 拦截规则使用 `antMatchers`（`requestMatchers` 的部分重载自 5.8 起才提供）；方法级安全使用 `@EnableGlobalMethodSecurity(prePostEnabled = true)` 开启；`Security 5.7` 已提供 `AuthorizationManager` 授权 API，任务二的自定义权限管理器同样按此实现；`AuthenticationManager` 通过 `@Bean` 暴露供登录接口调用。

截图：（安全配置类）

```

config/SecurityConfig.java

1 package com.grbroadcast.config;
2
3 import com.grbroadcast.security.AccessDeniedHandlerImpl;
4 import com.grbroadcast.security.AuthenticationEntryPointImpl;
5 import com.grbroadcast.security.JwtAuthenticationTokenFilter;
6 import com.grbroadcast.security.PermissionManager;
7 import org.springframework.context.annotation.Bean;
8 import org.springframework.context.annotation.Configuration;
9 import org.springframework.security.authentication.AuthenticationManager;
10 import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
11 import org.springframework.security.config.annotation.method.configuration.EnableGlobalMethodSecurity;
12 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
13 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
14 import org.springframework.security.config.http.SessionCreationPolicy;
15 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
16 import org.springframework.security.crypto.password.PasswordEncoder;
17 import org.springframework.security.web.SecurityFilterChain;
18 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
19 import org.springframework.web.cors.CorsConfiguration;
20 import org.springframework.web.cors.CorsConfigurationSource;
21 import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
22
23 /**
24  * Spring Security 安全配置 (Spring Boot 2.7 / Security 5.7)
25  */
26 @Configuration
27 @EnableWebSecurity
28 @EnableGlobalMethodSecurity(prePostEnabled = true)
29 public class SecurityConfig {
30
31     /** 白名单路径 */
32     private static final String[] WHITELIST = {
33         "/auth/login", "/user/register", "/error"
34     };
35
36     @Bean
37     public PasswordEncoder passwordEncoder() {
38         return new BCryptPasswordEncoder();
39     }
40
41     @Bean
42     public AuthenticationManager authenticationManager(AuthenticationConfiguration configuration)
43         throws Exception {
44         return configuration.getAuthenticationManager();
45     }
46
47     @Bean
48     public CorsConfigurationSource corsConfigurationSource() {
49         CorsConfiguration configuration = new CorsConfiguration();
50         configuration.addAllowedOriginPattern("*");
51         configuration.addAllowedHeader("*");
52         configuration.addAllowedMethod("*");
53         UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
54         source.registerCorsConfiguration("/**", configuration);
55         return source;
56     }
57
58     @Bean
59     public SecurityFilterChain filterChain(HttpSecurity http,
60         JwtAuthenticationTokenFilter jwtAuthenticationTokenFilter,
61         AuthenticationEntryPointImpl authenticationEntryPoint,
62         AccessDeniedHandlerImpl accessDeniedHandler,
63         PermissionManager permissionManager) throws Exception {
64
65         http
66             // 关闭 CSRF: 前后端分离使用令牌认证
67             .csrf().disable()
68             // 跨域交由安全框架统一处理
69             .cors().configurationSource(corsConfigurationSource())
70             .and()
71             // 关闭表单登录与 HTTP 基础认证
72             .formLogin().disable()
73             .httpBasic().disable()
74             // 关闭框架自带退出接口, 自行实现退出逻辑
75             .logout().disable()
76             // 不创建 HttpSession, 无状态
77             .sessionManagement().sessionCreationPolicy(SessionCreationPolicy.STATELESS)
78             .and()
79             // 401 与 403 分流处理
80             .exceptionHandling()
81             .authenticationEntryPoint(authenticationEntryPoint)
82             .accessDeniedHandler(accessDeniedHandler)
83             .and()
84             .authorizeHttpRequests(auth -> auth
85                 // 预检请求放行
86                 .antMatchers(org.springframework.http.HttpMethod.OPTIONS, "/**").permitAll()
87                 // 白名单放行
88                 .antMatchers(WHITELIST).permitAll()
89                 // 管理类路径走自定义权限管理器
90                 .antMatchers("/song/pending", "/article/pending",
91                     "/audio/upload", "/audio/delete/**",
92                     "/message/reply/**",

```

```

92         "/program/publish", "/program/update", "/program/delete/**",
93         "/menu/**")
94         .access(permissionManager)
95         // 其余请求要求已认证
96         .anyRequest().authenticated())
97         // JWT 过滤器置于用户名密码认证过滤器之前
98         .addFilterBefore(jwtAuthenticationTokenFilter,
99             UsernamePasswordAuthenticationFilter.class);
100     return http.build();
101 }
102 }
103

```

图8 SecurityConfig 安全配置类

步骤6：实现登录接口

新增登录控制器与登录业务类，业务方法按七步实现：

接收账号密码参数，做基本格式校验。

构造一个未认证的认证请求对象，把账号密码装进去。

调用认证管理器完成认证，框架会自动调用步骤 4 编写的用户详情服务。

从认证结果中取出用户详情对象，此时它已装配好角色与权限集合。

把用户详情对象以用户 ID 为键写入 Redis，过期时间与令牌有效期保持一致。

签发令牌，载荷中只放用户 ID。

组装响应：令牌字符串、有效期秒数，以及供前端使用的用户信息（账号、姓名、角色、权限标识列表）。

注意：认证失败时要统一转成「账号或密码错误」，不要分别提示「账号不存在」和「密码错误」，否则提示差异会被用来探测账号是否存在。

截图：（Apifox 调用登录接口，成功返回令牌）

```

controller/AuthController.java

1 package com.grbroadcast.controller;
2
3 import com.grbroadcast.common.Result;
4 import com.grbroadcast.security.LoginUser;
5 import com.grbroadcast.utils.JwtUtil;
6 import com.grbroadcast.utils.RedisUtil;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.security.authentication.AuthenticationManager;
9 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.security.core.AuthenticationException;
12 import org.springframework.security.core.context.SecurityContextHolder;
13 import org.springframework.util.StringUtils;
14 import org.springframework.web.bind.annotation.*;
15
16 import java.util.ArrayList;
17 import java.util.HashMap;
18 import java.util.List;
19 import java.util.Map;
20
21 /**
22  * 认证控制器：登录（签发 JWT）、退出（删除缓存）、当前用户信息
23  */
24 @RestController
25 @RequestMapping("/auth")
26 public class AuthController {
27
28     private static final String CACHE_PREFIX = "login:user:";
29
30     @Autowired
31     private AuthenticationManager authenticationManager;
32
33     @Autowired
34     private RedisUtil redisUtil;
35
36     @Autowired
37     private JwtUtil jwtUtil;
38
39     /** 登录 */
40     @PostMapping("/login")
41     public Result login(@RequestBody Map<String, String> body) {
42         String username = body.get("username");
43         String password = body.get("password");
44         // 1. 基本格式校验
45         if (!StringUtils.hasText(username) || !StringUtils.hasText(password)) {
46             return Result.error("请输入账号和密码");
47         }
48         // 2/3. 构造认证请求并交由认证管理器完成认证
49         Authentication authentication;
50         try {
51             authentication = authenticationManager.authenticate(
52                 new UsernamePasswordAuthenticationToken(username, password));
53         } catch (AuthenticationException e) {
54             // 统一提示，避免通过提示差异探测账号
55             return Result.error("账号或密码错误");
56         }
57         // 4. 取出已装配角色与权限的用户详情
58         LoginUser loginUser = (LoginUser) authentication.getPrincipal();
59         // 5. 以用户 ID 为键写入 Redis，过期时间与令牌有效期一致
60         redisUtil.set(CACHE_PREFIX + loginUser.getUserId(), loginUser, jwtUtil.get_expire());
61         // 6. 签发令牌，载荷中只放用户 ID
62         String token = jwtUtil.create(loginUser.getUserId());
63         // 7. 组装响应
64         Map<String, Object> info = new HashMap<>();
65         info.put("token", token);
66         info.put("expire", jwtUtil.get_expire());
67         Map<String, Object> userInfo = new HashMap<>();
68         userInfo.put("userId", loginUser.getUserId());
69         userInfo.put("username", loginUser.getUsername());
70         userInfo.put("roles", loginUser.getRoles());
71         userInfo.put("permissions", loginUser.getPermissions());
72         info.put("user", userInfo);
73         return Result.success("登录成功", info);
74     }
75
76     /** 退出登录：删除 Redis 缓存并清空安全上下文 */
77     @PostMapping("/logout")
78     public Result logout() {
79         Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
80         if (authentication != null && authentication.getPrincipal() instanceof LoginUser) {
81             LoginUser loginUser = (LoginUser) authentication.getPrincipal();
82             redisUtil.delete(CACHE_PREFIX + loginUser.getUserId());
83         }
84         SecurityContextHolder.clearContext();
85         return Result.success("退出成功");
86     }
87
88     /** 获取当前登录用户信息 */
89     @GetMapping("/info")
90     public Result info() {
91         Authentication authentication = SecurityContextHolder.getContext().getAuthentication();

```



```

security/JwtAuthenticationTokenFilter.java

1 package com.grbroadcast.security;
2
3 import com.grbroadcast.utils.JwtUtil;
4 import com.grbroadcast.utils.RedisUtil;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.security.authentication.AnonymousAuthenticationToken;
7 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
8 import org.springframework.security.core.context.SecurityContextHolder;
9 import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
10 import org.springframework.stereotype.Component;
11 import org.springframework.util.StringUtils;
12 import org.springframework.web.filter.OncePerRequestFilter;
13
14 import javax.servlet.FilterChain;
15 import javax.servlet.ServletException;
16 import javax.servlet.http.HttpServletRequest;
17 import javax.servlet.http.HttpServletResponse;
18 import java.io.IOException;
19
20 /**
21  * JWT 认证过滤器：解析令牌 -> 从 Redis 取回用户详情 -> 写入安全上下文
22  */
23 @Component
24 public class JwtAuthenticationTokenFilter extends OncePerRequestFilter {
25
26     private static final String CACHE_PREFIX = "login:user:";
27
28     @Autowired
29     private RedisUtil redisUtil;
30
31     @Autowired
32     private JwtUtil jwtUtil;
33
34     @Override
35     protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
36                                     FilterChain chain) throws ServletException, IOException {
37         // 1. 从请求头取出令牌（兼容 Bearer 前缀与不带前缀）
38         String token = request.getHeader("Authorization");
39         if (StringUtils.hasText(token) && token.startsWith("Bearer ")) {
40             token = token.substring(7);
41         }
42         // 2. 无令牌直接放行，交由授权环节统一处理
43         if (!StringUtils.hasText(token)) {
44             chain.doFilter(request, response);
45             return;
46         }
47         // 3. 校验令牌签名与有效期
48         if (!jwtUtil.verify(token)) {
49             chain.doFilter(request, response);
50             return;
51         }
52         // 4. 解析用户 ID
53         Long userId = jwtUtil.getUserId(token);
54         // 5. 从 Redis 取回用户详情
55         LoginUser loginUser = (LoginUser) redisUtil.get(CACHE_PREFIX + userId);
56         // 6. 缓存中不存在则放行
57         if (loginUser == null) {
58             chain.doFilter(request, response);
59             return;
60         }
61         // 7. 二次校验账号状态
62         if (!loginUser.isEnabled()) {
63             chain.doFilter(request, response);
64             return;
65         }
66         // 8. 上下文为空或为匿名身份时，构造已认证对象写入安全上下文
67         org.springframework.security.core.Authentication existing =
68             SecurityContextHolder.getContext().getAuthentication();
69         if (existing != null && !(existing instanceof AnonymousAuthenticationToken)) {
70             chain.doFilter(request, response);
71             return;
72         }
73         UsernamePasswordAuthenticationToken authentication =
74             new UsernamePasswordAuthenticationToken(loginUser, null, loginUser.getAuthorities());
75         authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
76         SecurityContextHolder.getContext().setAuthentication(authentication);
77         chain.doFilter(request, response);
78     }
79 }
80

```

图11 JwtAuthenticationTokenFilter 认证过滤器

```
Windows PowerShell
==== 场景2: 携带令牌访问 /auth/info ====
PS> curl.exe http://localhost:8089/api/auth/info -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJleHAiOjE3OTZ0Z0Z0InVzZXJJZCI6NSwiaWF0IjoxNzIxMzQ1ODg4fQ.9mYxOyEUnI6_7ouHhrGsHPsd02UsvfJEirn0oRygbIE"
{
  "code": 200,
  "msg": "success",
  "data": {
    "permissions": [
      "article:edit",
      "audio:upload",
      "program:list",
      "article:audit",
      "article:add",
      "song:audit",
      "audio:delete",
      "message:add",
      "menu:list",
      "song:add",
      "audio:list",
      "program:publish",
      "article:list",
      "message:reply",
      "program:edit",
      "song:delete",
      "article:delete",
      "song:list",
      "menu:grant",
      "message:list",
      "song:edit",
      "program:delete"
    ],
    "userId": 5,
    "username": "staff1"
  }
}
PS C:\WINDOWS\System32>
```

图12 携带令牌访问 /auth/info 成功返回

步骤8: 实现退出登录

新增退出接口，按三步实现：从安全上下文取出当前已认证的用户信息、取出用户 ID 并按与登录时完全一致的规则拼出缓存键并删除、清空安全上下文后返回成功。

因为过滤器每次都从 Redis 取用户信息，所以缓存被删除后，该令牌即便签名有效、时间未到，也再取不到身份信息，等同于立即失效。

截图：（退出后用原令牌访问受保护接口，返回 401）

```
Windows PowerShell
==== 场景5: 退出登录后，原令牌立即失效返回 401 ====
PS> curl.exe -X POST http://localhost:8089/api/auth/logout -H "Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJleHAiOjE3OTZ0Z0Z0InVzZXJJZCI6NSwiaWF0IjoxNzIxMzQ1ODg4fQ.M2A0Ay6T27IZcuISWUthmChtl6UdB_r0py3dyqm60m0"
{"code":200,"msg":"退出成功","data":null}
PS> curl.exe http://localhost:8089/api/auth/info -H "Authorization: Bearer <原令牌>"
{"code":401,"msg":"未登录或登录已过期，请重新登录"}
HTTP/STATUS:401
PS C:\WINDOWS\System32>
```

图13 退出登录后原令牌立即失效返回 401

步骤9: 设计菜单表、菜单树与角色菜单授权

设计 menu 菜单表与 role_menu 角色菜单关联表。menu 表用 menu_type 字段区分目录 M、菜单 C、按钮 F 三类数据，权限标识按「模块:操作」命名，例如 song:audit、article:add；角色与菜单通过 role_menu 多对多关联。

菜单树采用两次遍历加映射表构建：第一次遍历把全部节点放入以菜单 ID 为键的映射表，第二次遍历为每个节点查找父节点并挂载、同时收集根节点。整体时间复杂度为 $O(n)$ ，既避免递归反复遍历列表，也避免层级过深时递归栈溢出。

角色菜单授权：角色列表接口返回各角色及其已选菜单；保存授权时在一个事务内先删除该角色的旧关联，再批量插入新关联，并自动补全所选菜单的父级目录，避免出现「能打开页面但上级目录不可见」。

```

service/impl/MenuServiceImpl.java

1 package com.grbroadcast.service.impl;
2
3 import com.grbroadcast.dao.MenuMapper;
4 import com.grbroadcast.entity.Menu;
5 import com.grbroadcast.service.MenuService;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.stereotype.Service;
8 import org.springframework.transaction.annotation.Transactional;
9
10 import java.util.ArrayList;
11 import java.util.Arrays;
12 import java.util.HashMap;
13 import java.util.HashSet;
14 import java.util.LinkedHashMap;
15 import java.util.List;
16 import java.util.Map;
17 import java.util.Set;
18
19 @Service
20 public class MenuServiceImpl implements MenuService {
21
22     @Autowired
23     private MenuMapper menuMapper;
24
25     private static final List<String> ROLES = Arrays.asList("staff", "teacher", "student");
26
27     @Override
28     public List<Menu> listAll() {
29         return menuMapper.selectList(null);
30     }
31
32     @Override
33     public List<Menu> treeAll() {
34         return buildTree(listAll());
35     }
36
37     @Override
38     public List<Menu> treeByRole(String roleCode) {
39         return buildTree(menuMapper.selectMenusByRole(roleCode));
40     }
41
42     @Override
43     public List<Map<String, Object>> listRolesWithMenus() {
44         List<Map<String, Object>> result = new ArrayList<>();
45         for (String role : ROLES) {
46             List<Menu> menus = menuMapper.selectMenusByRole(role);
47             List<Long> ids = new ArrayList<>();
48             for (Menu menu : menus) {
49                 ids.add(menu.getId());
50             }
51             Map<String, Object> item = new LinkedHashMap<>();
52             item.put("roleCode", role);
53             item.put("menuIds", ids);
54             result.add(item);
55         }
56         return result;
57     }
58
59     /**
60      * 两次遍历 + 映射表构建菜单树:
61      * 第一次遍历建立 id -> 菜单 映射并挂载子节点;
62      * 第二次遍历收集根节点。
63      * 不使用递归: 构建过程为 O(n) 线性处理, 避免递归深度限制与重复查找父节点,
64      * 平铺结果集只需线性扫描即可完成父子挂载。
65      */
66     @Override
67     public List<Menu> buildTree(List<Menu> menus) {
68         // 第一次遍历: 映射表 + 挂载子节点
69         Map<Long, Menu> idMap = new HashMap<>();
70         for (Menu menu : menus) {
71             menu.setChildren(new ArrayList<>());
72             idMap.put(menu.getId(), menu);
73         }
74         List<Menu> roots = new ArrayList<>();
75         for (Menu menu : menus) {
76             Long parentId = menu.getParentId();
77             if (parentId != null && idMap.containsKey(parentId)) {
78                 idMap.get(parentId).getChildren().add(menu);
79             } else {
80                 // 第二次 (收集根节点, 与挂载同遍历即时判定)
81                 roots.add(menu);
82             }
83         }
84         return roots;
85     }
86
87     /**
88      * 角色菜单授权: 事务保护, 先删后插;
89      * 勾选子菜单时自动补全其全部父级目录
90      */
91     @Override

```

```

92     @Transactional(rollbackFor = Exception.class)
93     public void grantRoleMenus(String roleCode, List<Long> menuIds) {
94         // 1. 先删除该角色的全部关联
95         menuMapper.deleteRoleMenu(roleCode);
96         if (menuIds == null || menuIds.isEmpty()) {
97             return;
98         }
99         // 2. 建立全部菜单映射, 用于回溯父级目录
100         Map<Long, Menu> allMap = new HashMap<>();
101         for (Menu menu : listAll()) {
102             allMap.put(menu.getId(), menu);
103         }
104         // 3. 补全父级目录后去重
105         Set<Long> toGrant = new HashSet<>(menuIds);
106         for (Long menuId : menuIds) {
107             Menu current = allMap.get(menuId);
108             while (current != null && current.getParentId() != null
109                 && current.getParentId() != 0L) {
110                 Menu parent = allMap.get(current.getParentId());
111                 if (parent == null) {
112                     break;
113                 }
114                 toGrant.add(parent.getId());
115                 current = parent;
116             }
117         }
118         // 4. 逐条插入
119         for (Long menuId : toGrant) {
120             menuMapper.insertRoleMenu(roleCode, menuId);
121         }
122     }
123 }
124

```

图14 MenuServiceImpl 菜单树构建与角色授权（先删后插、事务保护）

```

controller/MenuController.java

1 package com.grbroadcast.controller;
2
3 import com.grbroadcast.common.Result;
4 import com.grbroadcast.service.MenuService;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.security.access.prepost.PreAuthorize;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10 import java.util.Map;
11
12 /**
13  * 菜单与角色授权接口
14  */
15 @RestController
16 @RequestMapping("/menu")
17 public class MenuController {
18
19     @Autowired
20     private MenuService menuService;
21
22     /** 全部菜单（平铺） */
23     @GetMapping("/list")
24     @PreAuthorize("@ss.hasPermi('menu:list')")
25     public Result list() {
26         return Result.success(menuService.listAll());
27     }
28
29     /** 全部菜单树 */
30     @GetMapping("/tree")
31     @PreAuthorize("@ss.hasPermi('menu:list')")
32     public Result tree() {
33         return Result.success(menuService.treeAll());
34     }
35
36     /** 角色列表，携带各自菜单ID */
37     @GetMapping("/roles")
38     @PreAuthorize("@ss.hasPermi('menu:list')")
39     public Result roles() {
40         return Result.success(menuService.listRolesWithMenus());
41     }
42
43     /** 查询指定角色的菜单树 */
44     @GetMapping("/role/{roleCode}")
45     @PreAuthorize("@ss.hasPermi('menu:list')")
46     public Result roleMenus(@PathVariable String roleCode) {
47         return Result.success(menuService.treeByRole(roleCode));
48     }
49
50     /** 给指定角色授权菜单（先删后插、事务保护、父级目录补全） */
51     @PutMapping("/role/{roleCode}")
52     @PreAuthorize("@ss.hasPermi('menu:grant')")
53     public Result grant(@PathVariable String roleCode,
54                        @RequestBody Map<String, List<Long>> body) {
55         menuService.grantRoleMenus(roleCode, body.get("menuIds"));
56         return Result.success("授权成功");
57     }
58 }
59

```

图15 MenuController 菜单与授权接口

任务二：实现授权拦截（401 与 403）

步骤1：实现权限判断服务

新建权限判断服务类，注册为名为 ss 的 Bean，对外提供四个方法：判断是否拥有指定权限、判断是否拥有任意权限、判断是否拥有指定角色、判断是否为超级管理员。

判断逻辑分三步：从安全上下文中取出当前登录用户，取不到或权限集合为空则直接返回不通过；权限集合中含有通配符则说明是超级管理员，直接返回通过；否则判断权限集合是否包含目标权限标识。

路径级授权与后面的方法注解必须共用这一处逻辑，否则规则演进时两处会不一致。

截图：（权限判断服务类）

```

security/PermissionService.java

1 package com.grbroadcast.security;
2
3 import org.springframework.security.core.Authentication;
4 import org.springframework.security.core.context.SecurityContextHolder;
5 import org.springframework.stereotype.Service;
6
7 import java.util.Set;
8
9 /**
10  * 权限判断服务，注册为名为 ss 的 Bean，
11  * 路径级授权与方法级注解共用这一处判断逻辑
12  */
13 @Service("ss")
14 public class PermissionService {
15
16     private static final String ALL_PERMISSION = "*:*:*";
17
18     /** 判断是否拥有指定权限 */
19     public boolean hasPermi(String permission) {
20         LoginUser loginUser = getLoginUser();
21         if (loginUser == null || isEmpty(loginUser.getPermissions())) {
22             return false;
23         }
24         if (loginUser.getPermissions().contains(ALL_PERMISSION)) {
25             return true;
26         }
27         return loginUser.getPermissions().contains(permission);
28     }
29
30     /** 判断是否拥有任意一个权限 */
31     public boolean hasAnyPermi(String[] permissions) {
32         LoginUser loginUser = getLoginUser();
33         if (loginUser == null || isEmpty(loginUser.getPermissions())) {
34             return false;
35         }
36         if (loginUser.getPermissions().contains(ALL_PERMISSION)) {
37             return true;
38         }
39         for (String permission : permissions) {
40             if (loginUser.getPermissions().contains(permission)) {
41                 return true;
42             }
43         }
44         return false;
45     }
46
47     /** 判断是否拥有指定角色 */
48     public boolean hasRole(String role) {
49         LoginUser loginUser = getLoginUser();
50         return loginUser != null && !isEmpty(loginUser.getRoles())
51             && loginUser.getRoles().contains(role);
52     }
53
54     /** 判断是否为超级管理员 */
55     public boolean isAdmin() {
56         LoginUser loginUser = getLoginUser();
57         return loginUser != null && !isEmpty(loginUser.getPermissions())
58             && loginUser.getPermissions().contains(ALL_PERMISSION);
59     }
60
61     private LoginUser getLoginUser() {
62         Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
63         if (authentication == null || !authentication.isAuthenticated()) {
64             return null;
65         }
66         Object principal = authentication.getPrincipal();
67         return principal instanceof LoginUser ? (LoginUser) principal : null;
68     }
69
70     private boolean isEmpty(Set<?> set) {
71         return set == null || set.isEmpty();
72     }
73 }
74

```

图16 PermissionService 权限判断服务（Bean 名 ss）

步骤2：实现自定义权限管理器

新建权限管理器类，实现 `AuthorizationManager` 接口，泛型参数取请求授权上下文，表示基于请求的授权判断。判断过程分四层：

从请求上下文取出请求对象，得到请求方法与请求路径，并取出当前认证信息。

若认证信息为空、未通过认证，或主体是匿名用户，返回拒绝。

若当前用户是超级管理员，直接返回通过，不再逐条比对权限。

根据请求方法与路径解析出该请求所需的权限标识；若该路径没有配置权限要求，返回通过；否则调用权限判断服务比对，命中返回通过，未命中返回拒绝并记录告警日志。

路径与权限的对应关系建议集中在一处常量或数据库中维护，便于审阅「哪些接口受保护」。

截图：（权限管理器类）

```

security/PermissionManager.java

1 package com.grbroadcast.security;
2
3 import org.slf4j.Logger;
4 import org.slf4j.LoggerFactory;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.security.authentication.AnonymousAuthenticationToken;
7 import org.springframework.security.authorization.AuthorizationDecision;
8 import org.springframework.security.authorization.AuthorizationManager;
9 import org.springframework.security.core.Authentication;
10 import org.springframework.security.web.access.intercept.RequestAuthorizationContext;
11 import org.springframework.stereotype.Component;
12
13 import java.util.function.Supplier;
14
15 /**
16  * 自定义权限管理器：基于请求方法与路径解析所需权限标识并比对
17  */
18 @Component
19 public class PermissionManager implements AuthorizationManager<RequestAuthorizationContext> {
20
21     private static final Logger log = LoggerFactory.getLogger(PermissionManager.class);
22
23     @Autowired
24     private PermissionService permissionService;
25
26     @Override
27     public AuthorizationDecision check(Supplier<Authentication> authentication,
28                                     RequestAuthorizationContext context) {
29         // 1. 取出认证信息
30         Authentication auth = authentication.get();
31         // 2. 未认证或匿名用户，拒绝
32         if (auth == null || !auth.isAuthenticated()
33             || auth instanceof AnonymousAuthenticationToken) {
34             return new AuthorizationDecision(false);
35         }
36         // 3. 超级管理员直接通过
37         if (permissionService.isAdmin()) {
38             return new AuthorizationDecision(true);
39         }
40         // 4. 解析请求所需权限标识
41         String requiredPermission = resolvePermission(context);
42         if (requiredPermission == null) {
43             // 该路径没有配置权限要求，放行
44             return new AuthorizationDecision(true);
45         }
46         boolean granted = permissionService.hasPermi(requiredPermission);
47         if (!granted) {
48             log.warn("授权失败：用户[{}]访问[{}]，缺少权限[{}]",
49                 auth.getName(), context.getRequest().getMethod(),
50                 context.getRequest().getRequestURI(), requiredPermission);
51         }
52         return new AuthorizationDecision(granted);
53     }
54
55     /**
56      * 路径与权限对应关系，集中在一处维护
57      */
58     private String resolvePermission(RequestAuthorizationContext context) {
59         String method = context.getRequest().getMethod();
60         String uri = context.getRequest().getRequestURI();
61         // 去掉上下文路径
62         String path = uri.replace("/api", "");
63         if ("GET".equals(method) && path.equals("/song/pending")) {
64             return "song:audit";
65         }
66         if ("GET".equals(method) && path.equals("/article/pending")) {
67             return "article:audit";
68         }
69         if ("POST".equals(method) && path.equals("/audio/upload")) {
70             return "audio:upload";
71         }
72         if ("DELETE".equals(method) && path.startsWith("/audio/delete")) {
73             return "audio:delete";
74         }
75         if ("PUT".equals(method) && path.startsWith("/message/reply")) {
76             return "message:reply";
77         }
78         if ("POST".equals(method) && path.equals("/program/publish")) {
79             return "program:publish";
80         }
81         if ("PUT".equals(method) && path.equals("/program/update")) {
82             return "program:edit";
83         }
84         if ("DELETE".equals(method) && path.startsWith("/program/delete")) {
85             return "program:delete";
86         }
87         if ("GET".equals(method) && path.startsWith("/menu")) {
88             return "menu:list";
89         }
90         // 其余路径无专门权限要求
91         return null;
92     }
93 }

```

```
92     }
93 }
94
```

图17 PermissionManager 自定义权限管理器

步骤3：实现两个异常处理器

新建 handler 包，在其中创建两个处理器，均使用组件注解注册。

无权限异常处理器实现 AccessDeniedHandler 接口，重写 handle 方法：设置响应状态码为 403，设置响应内容类型与字符编码（避免中文乱码），把统一的失败响应对象序列化为 JSON 写入响应体。

认证异常处理器实现 AuthenticationEntryPoint 接口，重写 commence 方法：设置响应状态码为 401，其余处理同上。

两个对照关系如下。

对比项	认证异常处理器	无权限异常处理器
处理场景	未登录或凭证无效	已登录但权限不足
实现接口	AuthenticationEntryPoint	AccessDeniedHandler
重写方法	commence	handle
HTTP 状态码	401	403

两个处理器中都不要抛出异常，也不要继续调用过滤器链，否则响应会被重复写入。

截图：（无令牌访问受控接口返回 401；读者账号调用管理员接口返回 403）

```

security/AuthenticationEntryPointImpl.java

1 package com.grbroadcast.security;
2
3 import com.alibaba.fastjson.JSON;
4 import com.grbroadcast.common.Result;
5 import org.springframework.security.core.AuthenticationException;
6 import org.springframework.security.web.AuthenticationEntryPoint;
7 import org.springframework.stereotype.Component;
8
9 import javax.servlet.http.HttpServletRequest;
10 import javax.servlet.http.HttpServletResponse;
11 import java.io.IOException;
12
13 /**
14  * 认证异常处理器：未登录或凭证无效，返回 401
15  */
16 @Component
17 public class AuthenticationEntryPointImpl implements AuthenticationEntryPoint {
18
19     @Override
20     public void commence(HttpServletRequest request, HttpServletResponse response,
21         AuthenticationException e) throws IOException {
22         response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
23         response.setContentType("application/json;charset=UTF-8");
24         Result result = Result.error(401, "未登录或登录已过期，请重新登录");
25         response.getWriter().write(JSON.toJSONString(result));
26     }
27 }
28
29
30 security/AccessDeniedHandlerImpl.java
31
32 package com.grbroadcast.security;
33
34 import com.alibaba.fastjson.JSON;
35 import com.grbroadcast.common.Result;
36 import org.springframework.security.access.AccessDeniedException;
37 import org.springframework.security.web.access.AccessDeniedHandler;
38 import org.springframework.stereotype.Component;
39
40 import javax.servlet.http.HttpServletRequest;
41 import javax.servlet.http.HttpServletResponse;
42 import java.io.IOException;
43
44 /**
45  * 无权限异常处理器：已登录但权限不足，返回 403
46  */
47 @Component
48 public class AccessDeniedHandlerImpl implements AccessDeniedHandler {
49
50     @Override
51     public void handle(HttpServletRequest request, HttpServletResponse response,
52         AccessDeniedException e) throws IOException {
53         response.setStatus(HttpServletResponse.SC_FORBIDDEN);
54         response.setContentType("application/json;charset=UTF-8");
55         Result result = Result.error(403, "没有操作权限");
56         response.getWriter().write(JSON.toJSONString(result));
57     }
58 }

```

图18 401 与 403 两个异常处理器

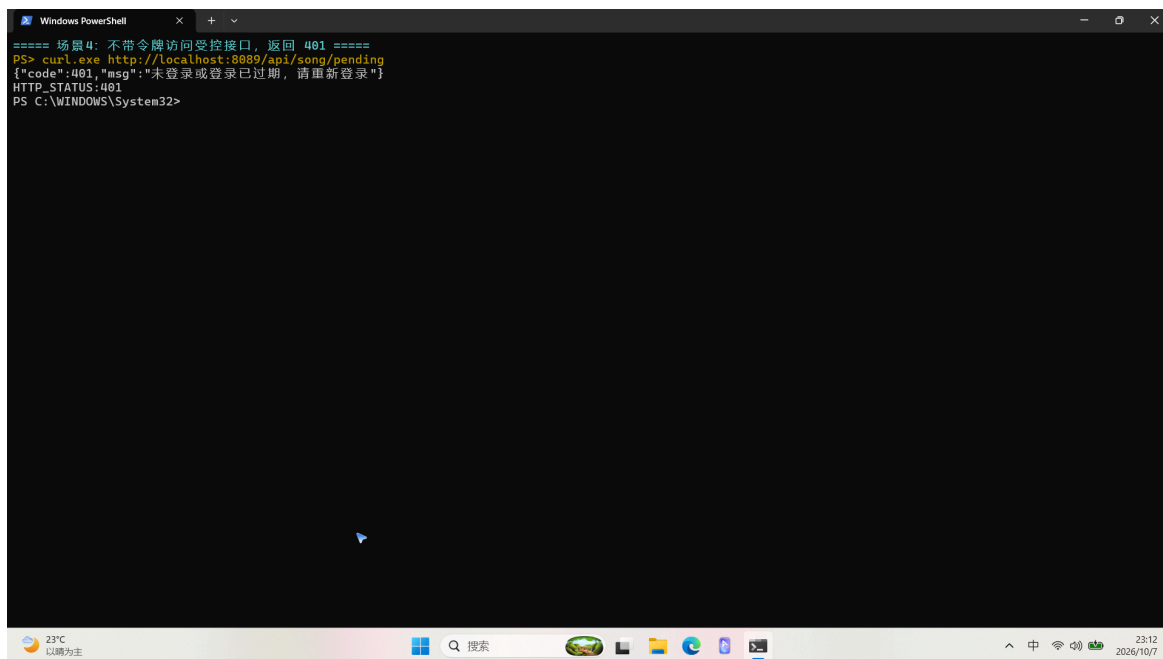


图19 无令牌访问受控接口返回 401

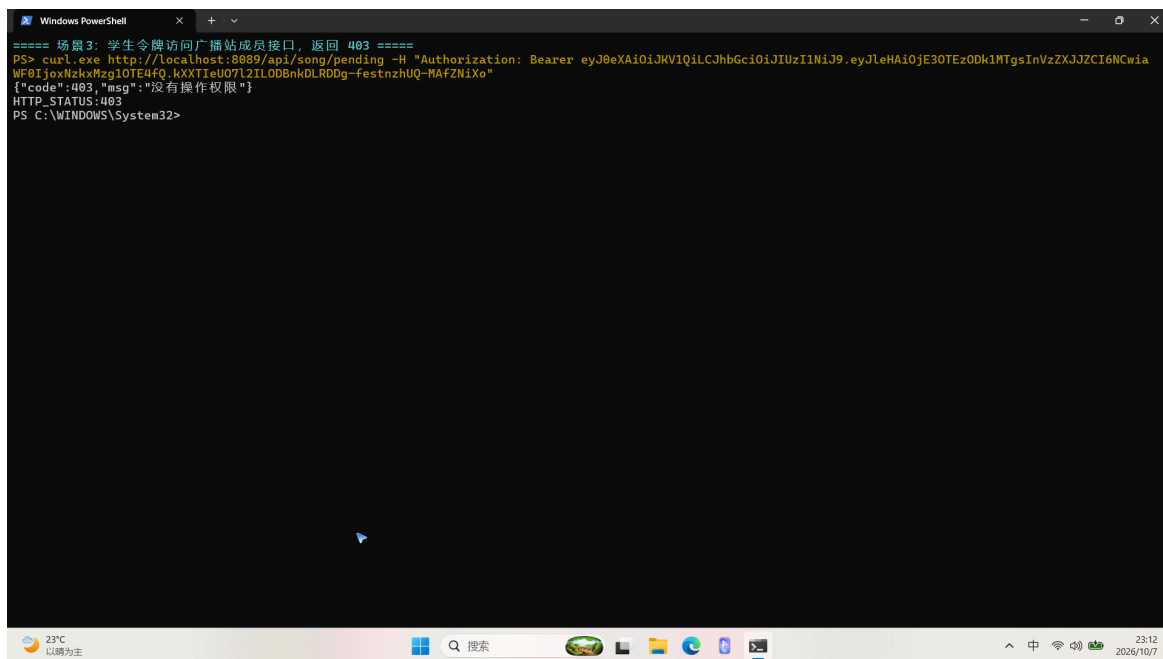


图20 学生账号调用广播站成员接口返回 403

步骤4：替换方法级权限注解

把原有 7 处「仅管理员」的判断逐一替换为具体权限注解。

原接口	原有判断	替换为
点歌模块接口	方法内 Session/角色判断	song:list、song:add、song:audit、song:edit、song:delete
投稿模块接口	方法内 Session/角色判断	article:list、article:add、article:audit、article:edit、article:delete
音频模块接口	登录即可访问	audio:list、audio:upload、audio:delete
留言模块接口	登录即可访问	message:list、message:add、message:reply
节目单模块接口	方法内角色判断	program:list、program:publish、program:edit、program:delete
菜单与授权接口	本次实验新增	menu:list、menu:grant

前置条件：安全配置类中必须已开启方法级安全注解，否则这些注解会静默失效，既不报错也不警告，现象是路径级拦截正常但注解完全不起作用。

截图：（替换后的控制器方法）

```
controller/SongController.java (权限注解)

1  @PreAuthorize("@ss.hasPermi('song:add')")
2  public Result addSong(@RequestBody SongRequest songRequest, @RequestParam Long userId) {
3      User user = userService.getById(userId);
4
5      @PreAuthorize("@ss.hasPermi('song:audit')")
6      public Result getPending() {
7          return Result.success(songService.lambdaQuery()
8
9      @PreAuthorize("@ss.hasPermi('song:audit')")
10     public Result auditSong(@PathVariable Long id,
11                             @RequestParam String status,
12
13     @PreAuthorize("@ss.hasPermi('song:delete')")
14     public Result deleteSong(@PathVariable Long id) {
15         return songService.deleteSong(id) ? Result.success("删除成功") : Result.error("删除失败");
16
17     @PreAuthorize("@ss.hasPermi('song:edit')")
18     public Result updateSong(@RequestBody SongRequest songRequest) {
19         return songService.updateById(songRequest) ? Result.success("修改成功") : Result.error("修改失败");
```

图21 SongController 方法级权限注解

```
controller/ArticleController.java (权限注解)

1  @PreAuthorize("@ss.hasPermi('article:add')")
2  public Result addArticle(@RequestBody Article article, @RequestParam Long userId) {
3      User user = userService.getById(userId);
4
5      @PreAuthorize("@ss.hasPermi('article:edit')")
6      public Result updateArticle(@RequestBody Article article) {
7          return articleService.updateById(article) ? Result.success("修改成功") : Result.error("修改失败");
8
9      @PreAuthorize("@ss.hasPermi('article:delete')")
10     public Result deleteArticle(@PathVariable Long id) {
11         return articleService.removeById(id) ? Result.success("删除成功") : Result.error("删除失败");
12
13     @PreAuthorize("@ss.hasPermi('article:audit')")
14     public Result auditArticle(@PathVariable Long id,
15                               @RequestParam String status,
16
17     @PreAuthorize("@ss.hasPermi('article:audit')")
18     public Result getPending() {
19         return Result.success(articleService.getPending());
```

图22 ArticleController 方法级权限注解

```
controller/AudioController.java (权限注解)

1  @PreAuthorize("@ss.hasPermi('audio:upload')")
2  public Result uploadAudio(@RequestParam("file") MultipartFile file,
3                             @RequestParam("title") String title,
4
5  @PreAuthorize("@ss.hasPermi('audio:delete')")
6  public Result deleteAudio(@PathVariable Long id) {
7      return audioService.removeById(id) ? Result.success("删除成功") : Result.error("删除失败");
```

图23 AudioController 方法级权限注解

```
controller/MessageController.java (权限注解)

1  @PreAuthorize("@ss.hasPermi('message:add')")
2  public Result addMessage(@RequestBody Message message, @RequestParam Long userId) {
3      User user = userService.getById(userId);
4
5  @PreAuthorize("@ss.hasPermi('message:reply')")
6  public Result deleteMessage(@PathVariable Long id) {
7      return messageService.removeById(id) ? Result.success("删除成功") : Result.error("删除失败");
8
9  @PreAuthorize("@ss.hasPermi('message:reply')")
10 public Result replyMessage(@PathVariable Long id,
11                             @RequestParam String reply,
```

图24 MessageController 方法级权限注解

```
controller/ProgramController.java (权限注解)

1  @PreAuthorize("@ss.hasPermi('program:publish')")
2  public Result publish(@RequestBody ProgramSchedule program, @RequestParam Long userId) {
3      User user = userService.getById(userId);
4
5  @PreAuthorize("@ss.hasPermi('program:delete')")
6  public Result deleteProgram(@PathVariable Long id) {
7      return programService.removeById(id) ? Result.success("删除成功") : Result.error("删除失败");
8
9  @PreAuthorize("@ss.hasPermi('program:edit')")
10 public Result updateProgram(@RequestBody ProgramSchedule program) {
11     return programService.updateById(program) ? Result.success("修改成功") : Result.error("修改失败");
```

图25 ProgramController 方法级权限注解